

Content Negotiation in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

Content Negotiation in ASP.NET Core Web API

In this article, I will discuss how to implement **Content Negotiation in an ASP.NET Core Web API** Application with Examples. Please read our previous article on how to apply the [Binding Attributes to Model Properties in an ASP.NET Core Web API](#) with Examples.

Content Negotiation is one of the fundamental RESTful principles that ensures a Web API can deliver responses in a format that clients can process and prefer. In ASP.NET Core Web API, this feature is automatically managed by the framework, allowing developers to support multiple response formats such as **JSON**, **XML**, or even **Custom Formats** like CSV or YAML.

In simpler terms, **Content Negotiation** is the “**Conversation**” between the client and the server about **Which Data Format** should be used for exchanging information. It improves **Flexibility**, **Interoperability**, and **Client Compatibility**.

Why Content Negotiation in RESTful Services?

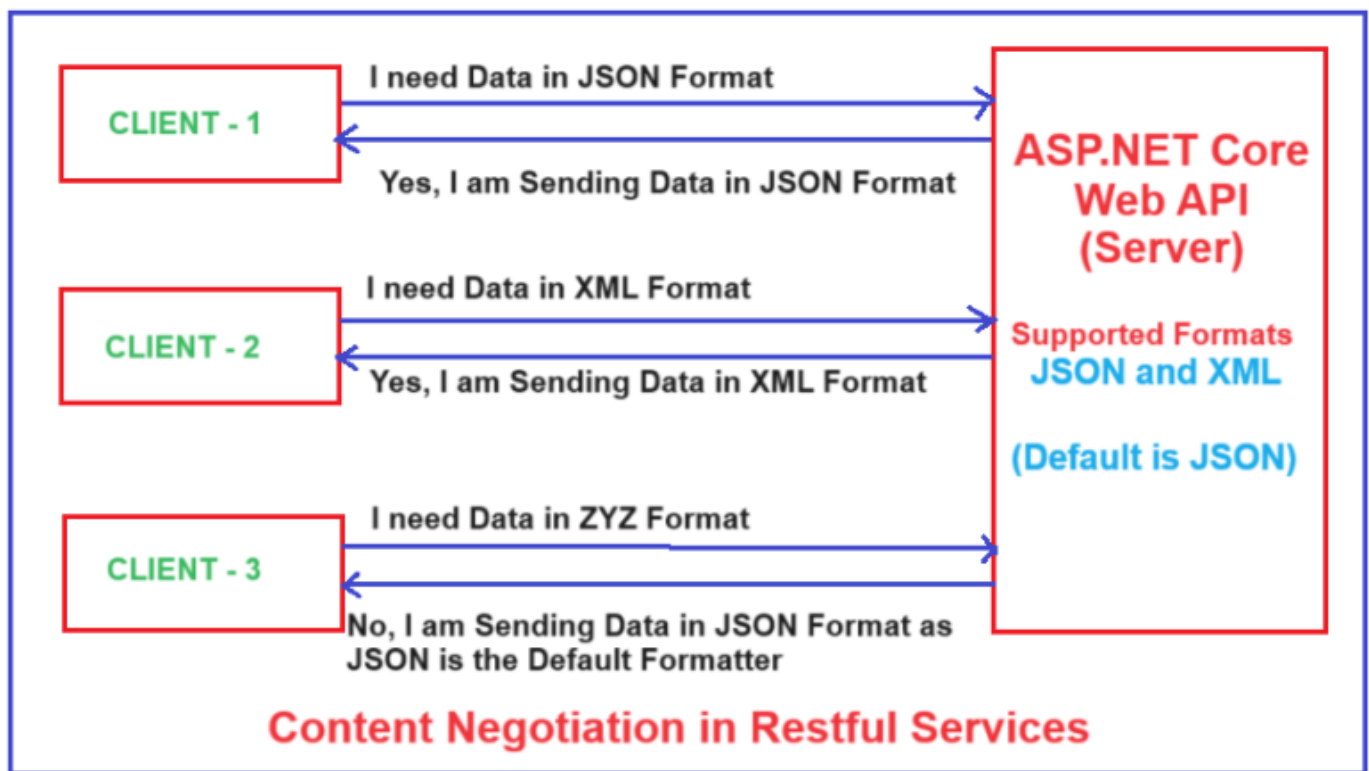
Content Negotiation provides the flexibility needed in modern distributed systems. Different clients, such as browsers, mobile apps, or legacy systems, may require data in different formats. ASP.NET Core allows the same API endpoint to serve data in multiple formats depending on client preferences.

Key Benefits

1. **Client Flexibility:** A single API can serve multiple clients with different format preferences. For example, Modern web apps prefer JSON, while old enterprise systems may require XML.
2. **Interoperability:** Supporting multiple data formats ensures that a wider range of clients can interact with your API without compatibility issues.
3. **Performance Optimization:** Clients can choose the most efficient format for their environment. JSON is smaller and faster over the network, while XML might be preferable for structured or schema-based communication.

Real-Time Example Scenario:

Assume your API supports two response formats, **JSON (default)** and **XML**. Here's how different clients interact with your API. For a better understanding, please have a look at the following diagram:



Example Scenarios

- **Client 1:** Requests data in JSON format via the **Accept: application/json** header. The server responds with JSON.
- **Client 2:** Requests data in XML format via the **Accept: application/xml** header. The server responds with XML.
- **Client 3:** Requests data in an unsupported format, say `application/xyz`. The server defaults to JSON, the primary format, or returns a 406 Not Acceptable status if configured to enforce strict content negotiation.

What is Content Negotiation?

Content Negotiation is the mechanism that enables a server to **select the best possible representation of a resource** based on the client's preferences.

Key HTTP Headers Involved:

1. **Accept:** Sent by the client to specify the media types it can handle, e.g., `Accept: application/json, application/xml`
2. **Content-Type:** Set by the server to indicate the media type of the response, e.g., `Content-Type: application/json`

ASP.NET Core uses **Output Formatters** to handle this process. Each formatter knows how to serialize an object into a specific format, such as JSON or XML. If no match is found, the framework either:

- Falls back to the default (usually JSON), or

- Returns 406 Not Acceptable if strict mode is enabled.

How Do We Implement Content Negotiation in ASP.NET Core Web API?

ASP.NET Core simplifies content negotiation through **Output Formatters**, which are automatically registered by calling `AddControllers()` in `Program.cs`. JSON is enabled by default, but XML and other formatters can be added easily.

Formatters in ASP.NET Core

ASP.NET Core uses two kinds of formatters:

Input Formatters

Handle **Deserialization** of the request body into .NET objects. Examples:

- `SystemTextJsonInputFormatter`
- `XmlSerializerInputFormatter`

Output Formatters

Handle **Serialization** of your response objects into the desired media type. Examples:

- `SystemTextJsonOutputFormatter`
- `XmlSerializerOutputFormatter`

When we call `AddControllers()`, ASP.NET Core registers a default list of formatters internally. By default, JSON formatters are registered first, and additional ones (like XML) can be added using `.AddXmlSerializerFormatters()`.

Example to Understand Content Negotiation in ASP.NET Core Web API:

Let us look at some examples to understand the concept of Content Negotiation in ASP.NET Core Web API. First, create a new ASP.NET Core Web API project named **ContentNegotiationDemo**. Then, create a folder named **Models** in the project root directory.

Create the Employee Model:

Create a class file named **Employee.cs** within the **Models** folder, and copy-paste the following code. This model acts as our resource representation that can be returned in both JSON and XML formats.

```
namespace ContentNegotiationDemo.Models
{
```

```

public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; } = null!;
    public string Gender { get; set; } = null!;
    public int Age { get; set; }
    public string Department { get; set; } = null!;
}

```

Create the Employee Controller:

Next, create an API Empty Controller named **EmployeeController** within the **Controllers** folder, and copy-paste the following code. Here, we have one action method that returns the list of employees.

```

using Microsoft.AspNetCore.Mvc;
using ContentNegotiationDemo.Models;

namespace ContentNegotiationDemo.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        //GET api/employee
        [HttpGet]
        public ActionResult<List<Employee>> GetEmployees()
        {
            var listEmployees = new List<Employee>()
            {
                new Employee(){ Id = 1001, Name = "Anurag", Age = 28, Gender = "Male", Department = "IT" },
                new Employee(){ Id = 1002, Name = "Pranaya", Age = 28, Gender = "Male", Department = "IT" },
            };

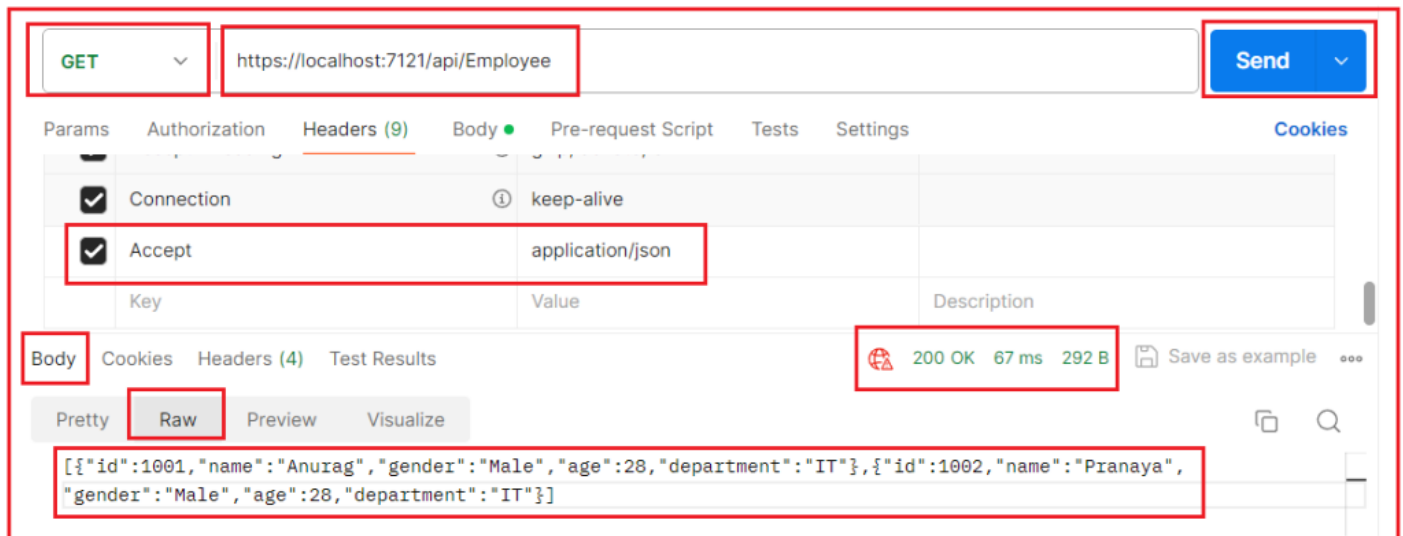
            return Ok(listEmployees);
        }
    }
}

```

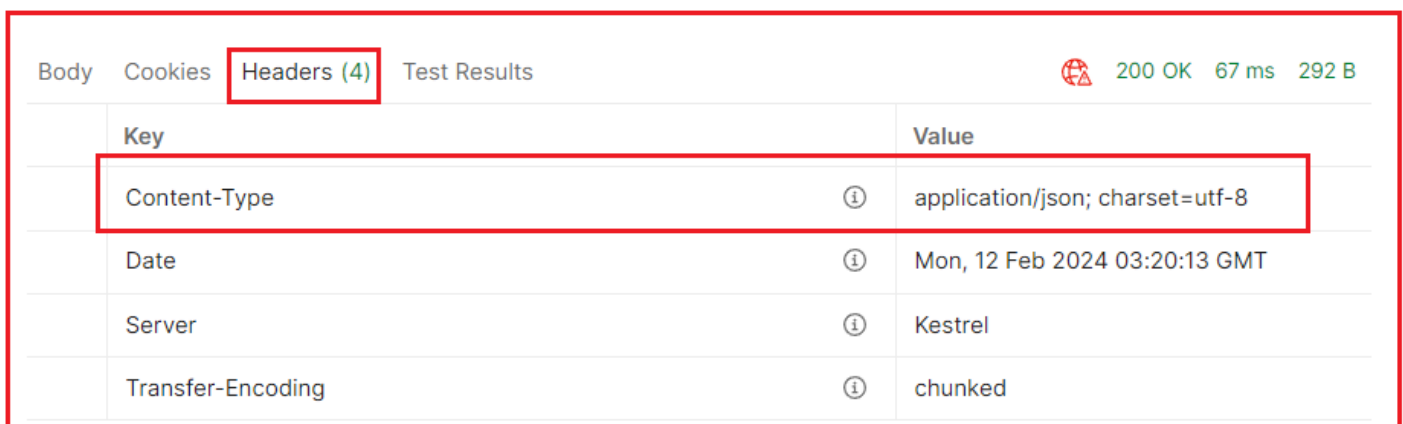
Test JSON Response:

Now, let us send an HTTP GET Request to the above endpoint using the Accept header and setting its value to **application/json**, telling the server to send the response in JSON format, as

shown in the image below. Please change the Port number where your application is running.

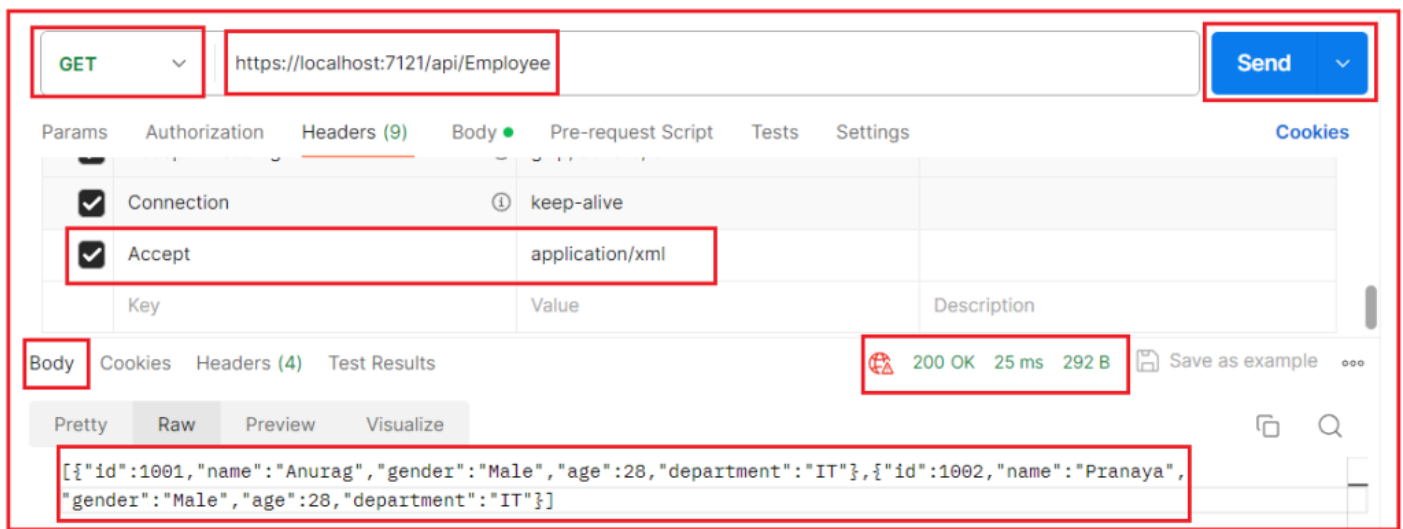


As you can see, we are getting the response in JSON format instead of XML. Now, if you verify the response header, then you will see it also sets the **Content-Type** header and sets its value to **application/json** as shown in the image below, as the server returns the response in JSON format:



Test XML Response:

Now, let us issue another HTTP GET Request to the above endpoint by setting the **Accept** header value to **application/xml**, as shown in the image below, and observe the format of the response.



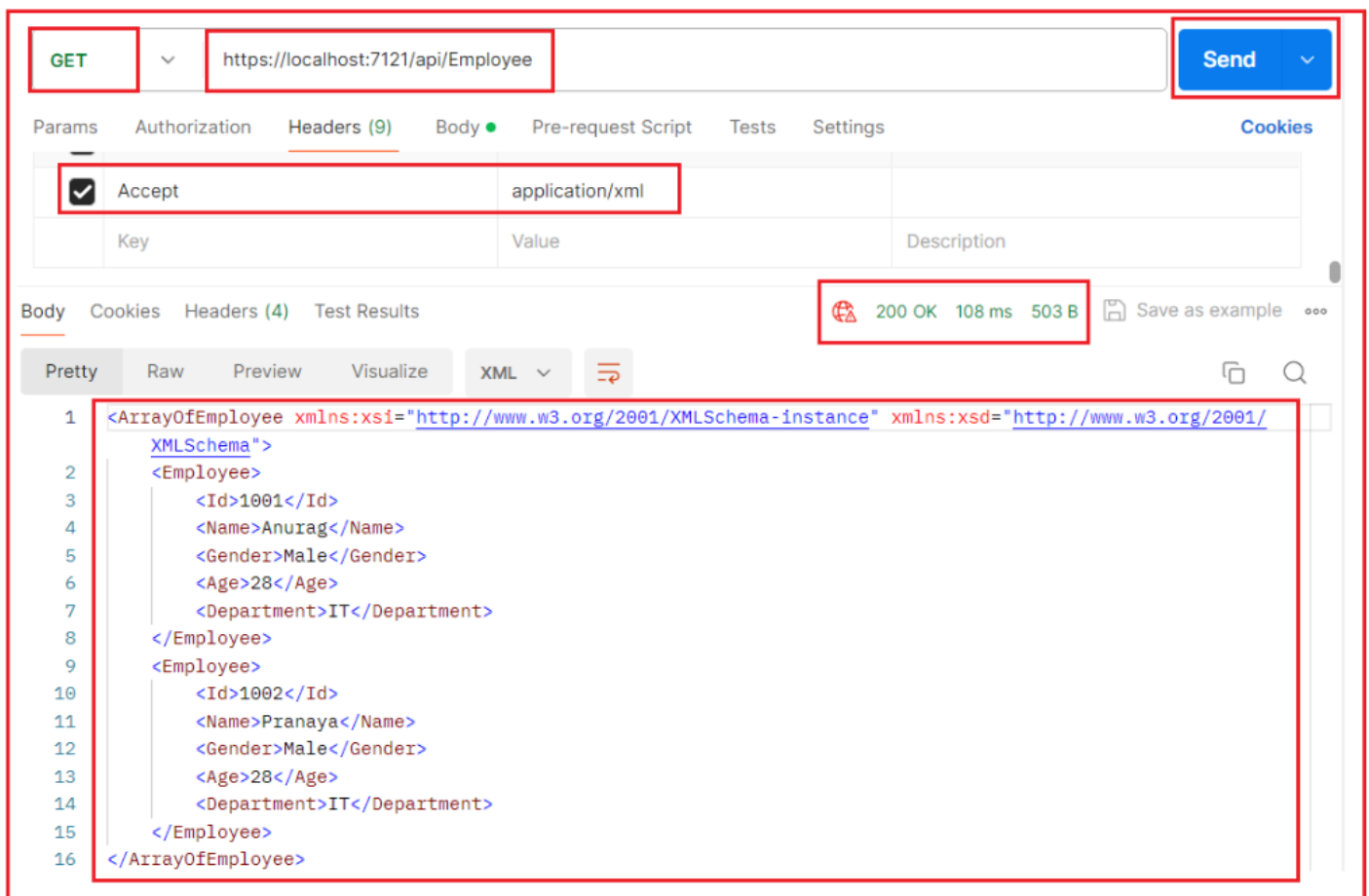
As you can see in the above image, we set the **Accept** header value to **application/xml**, but we get the response in XML format. This is because we have not yet enabled the XML formatter in our application. The only Formatter enabled by default is JSON Formatter, which is used to send the response irrespective of the Accept header value. Even if you check the response header, the Content-Type header value is set to application/json.

How to Enable XML Formatter in ASP.NET Core Web API?

We need to register the XML Formatter service with the dependency injection container, and we can do this in the Program class while registering the **AddControllers** services as follows:

```
builder.Services.AddControllers().AddXmlSerializerFormatters();
```

Here, the **AddXmlSerializerFormatters()** service enabled the XML Formatter for our application. These changes allow both XML and default JSON Formatters to be used for content negotiation. Now, if you send an HTTP GET Request with the Accept Header value set to **application/xml**, then you will get the response in XML format as shown in the image below:



Note: If you send an HTTP GET Request without specifying the Accept header value, you will get the response in JSON format as the default output formatter is JSON in ASP.NET Core Web API.

How to Make XML the Default Formatter?

To **make XML the default output formatter** in our ASP.NET Core Web API, we need to modify how output formatters are ordered and how the framework prioritizes them. By default, **JSON** is always the first formatter that ASP.NET Core tries, so even if XML is enabled, JSON will win unless explicitly requested otherwise.

Let us see how we can make **XML the default** response format. You need to change the order of formatters in the MVC options so that the **XML formatter is tried first**. Here, we need to register the XML formatter as the first formatter in the OutputFormatters collection.

```
using Microsoft.AspNetCore.Mvc.Formatters;
builder.Services.AddControllers(options =>
{
    // Move XML formatter to the top of the list to make it default
    var xmlFormatter = new XmlSerializerOutputFormatter();
    options.OutputFormatters.Insert(0, xmlFormatter);
});
```

By inserting the XML formatter at index 0, we make it the **highest priority** formatter during content negotiation. Now, if the client doesn't specify an Accept header (e.g., a browser or simple tool like curl), ASP.NET Core will choose the **first formatter** available. Since we placed XML first, it becomes the **default** format.

Multiple Accept Headers

While sending an HTTP Request, it is possible to set multiple values separated by a comma for the Accept header. In that case, the first value will take priority. That means if the client specifies both **application/json** and **application/xml** in the Accept header, the server will choose the first media type. For instance:

- If the client sends an **Accept: application/json, application/xml** header, and if the server supports JSON, JSON will be preferred.
- If the order is reversed, **Accept: application/xml, application/json**, XML will be preferred if the server supports it.

For example, set the Accept header value as **application/xml,application/json**. In that case, the server will give priority to XML and will return the data in XML Format, as shown in the image below:

The screenshot shows the Postman interface for an HTTP GET request to `https://localhost:7121/api/Employee`. The **Accept** header is set to `application/xml,application/json`. The response status is **200 OK** with a response time of **15 ms** and a body size of **503 B**. The response body is displayed in XML format, showing an array of two employee objects.

```
1 <ArrayOfEmployee xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
2   <Employee>
3     <Id>1001</Id>
4     <Name>Anurag</Name>
5     <Gender>Male</Gender>
6     <Age>28</Age>
7     <Department>IT</Department>
8   </Employee>
9   <Employee>
10    <Id>1002</Id>
11    <Name>Pranaya</Name>
12    <Gender>Male</Gender>
13    <Age>28</Age>
14    <Department>IT</Department>
15  </Employee>
16 </ArrayOfEmployee>
```

How to Remove the JSON Serializer in ASP.NET Core Web API?

JSON Formatter is registered by default in ASP.NET Core, as it is the most preferred approach for sending and receiving data nowadays. To remove the JSON Formatter in ASP.NET Core Web API, we need to modify the `AddControllers` method call to customize the options and remove the JSON Formatter as follows:

```
builder.Services.AddControllers(options =>
{
    // Remove JSON formatter

options.OutputFormatters.RemoveType<Microsoft.AspNetCore.Mvc.Formatters.SystemTextJsonOutputFormatter>();
}).AddXmlSerializerFormatters(); //Adding XML Formatter
```

With the above changes in place, the JSON Formatter is removed, and only the XML Formatter is registered to return the Response in XML Format. Now, send an HTTP Request, and the server will return the response in XML format irrespective of the Accept header value.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** https://localhost:7112/api/Employee
- Headers:** Accept: application/xml;q=0.5,application/json;q=0.8
- Status:** 200 OK, Time: 8 ms, Size: 503 B
- Body:** XML format, containing an `ArrayOfEmployee` schema with two employee entries.

```
<ArrayOfEmployee xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <Employee>
    <Id>1001</Id>
    <Name>Anurag</Name>
    <Gender>Male</Gender>
    <Age>28</Age>
    <Department>IT</Department>
  </Employee>
  <Employee>
    <Id>1002</Id>
    <Name>Pranaya</Name>
    <Gender>Male</Gender>
    <Age>28</Age>
    <Department>IT</Department>
  </Employee>
</ArrayOfEmployee>
```

How to Return 406 Not Acceptable Status Code if the Accepted Formatter is Not Available?

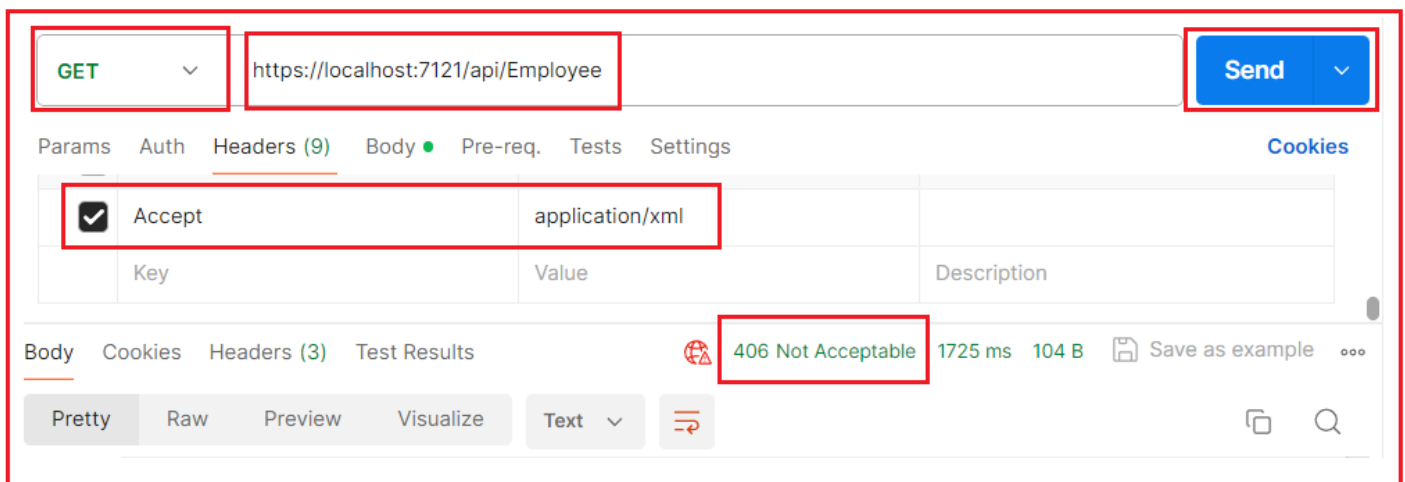
We need to return **406 Not Acceptable Status Code** when the server cannot produce a response based on the HTTP Request Accept header value. This scenario occurs when the client specifies an Accept header for a media type (e.g., `application/xml`, `application/json`) that the server cannot return.

However, by default, ASP.NET Core does not return the **406 Not Acceptable Status Code**. Instead, it returns the data using the default configured formatter. To force ASP.NET Core to return a **406 Not Acceptable Status Code** when the requested format is not supported, we need to configure the **ReturnHttpNotAcceptable** option and set its value to true while registering the AddControllers service.

So, modify the AddControllers service as follows. Here, we are using the default JSON Formatter. We removed the XML formatter and also configured it to ignore CamelCase. We are also explicitly setting to return 406 Not Acceptable Status Code:

```
builder.Services.AddControllers(options =>
{
    // Enable 406 Not Acceptable status code
    options.ReturnHttpNotAcceptable = true;
})
// Optionally, configure JSON options or other formatter settings
.AddJsonOptions(options =>
{
    // Configure JSON serializer settings if needed
    options.JsonSerializerOptions.PropertyNamingPolicy = null;
});
```

With the above changes in place, only the **Accept header** with **application/json** will work, and the data will be returned in **JSON format**. On the other hand, if you try to send an HTTP Request with an Accept header value set to **application/xml**, then you will get a 406 Not Acceptable Status Code, as shown in the image below:



Scenarios to Test:

- If you don't specify the Accept header value, by default, it will return the data in JSON Format.
- If you specify the Accept header value as **/*/***, accept all headers, it will return the data in the default JSON Format.
- If you specify the Accept header value as **application/json**, it will return the data in JSON Format.

- If you specify the Accept header value as application/xml, it will return a 406 Not Acceptable status code.
- If you specify the Accept header value to both XML and JSON, application/xml,application/json, it will return the data in JSON Format.

How to Customize 406 Status Code in ASP.NET Core Web API?

To customize the 406 Not Acceptable Status Code, we need to create a custom middleware component.

What is Middleware in ASP.NET Core?

Middleware is a software component that is executed as part of the **HTTP request-response pipeline** in an ASP.NET Core application. Each middleware component:

1. **Receives an incoming HTTP request.**
2. **Performs some processing** (e.g., logging, authentication, validation, etc.).
3. **Optionally passes the request** to the next middleware in the pipeline.
4. **May modify the outgoing response** before sending it to the client.

In simple terms, middleware acts as a **chain of request handlers**, where each component can **inspect**, **modify**, or **short-circuit** the request.

Key Characteristics of Middleware

- Middleware components are **Executed Sequentially** in the order they are added in Program.cs or Startup.cs.
- Each middleware decides **whether to Pass Control** to the next middleware using: **await _next(context);**
- **Middleware can read and modify both the HttpContext.Request and HttpContext.Response** objects.
- A middleware can **end the Request Early** (e.g., for invalid tokens, blocked IPs, etc.) without calling the next component. This is called **short-circuiting** the request.

Creating a Custom Middleware Component

Every middleware component must have:

- A **Constructor** that accepts a RequestDelegate.
- An **Invoke()** or **InvokeAsync()** method that accepts an HttpContext and returns a Task.

So, create a class file named **CustomNotAcceptableMiddleware.cs** within the **Models** folder and then copy-paste the following code. The following code is self-explained, so please read the comment lines for a better understanding.

```

// Importing the namespace for working with JSON serialization and
deserialization
using System.Text.Json;

namespace ContentNegotiationDemo.Models
{
    // A custom middleware class that handles '406 Not Acceptable' responses
    public class CustomNotAcceptableMiddleware
    {
        // Defining a field to store the next middleware in the pipeline
        private readonly RequestDelegate _next;

        // Constructor that takes in the next middleware in the pipeline
        public CustomNotAcceptableMiddleware(RequestDelegate next)
        {
            // Assigning the injected middleware to the private field
            _next = next;
        }

        // Middleware method called 'Invoke', which is executed for every HTTP
request
        public async Task Invoke(HttpContext context)
        {
            // Calling the next middleware in the pipeline and waiting for its
completion
            await _next(context);

            // Check if the response status code is 406 Not Acceptable
            if (context.Response.StatusCode ==
StatusCodes.Status406NotAcceptable)
            {
                // Retrieve the "Accept" header from the request, which indicates
the client's preferred response formats
                var acceptHeader = context.Request.Headers["Accept"].ToString();

                // Set the Content-Type of the response to 'application/json' to
indicate that the response is in JSON format
                context.Response.ContentType = "application/json";

                // Create an anonymous object containing the status code and a
custom error message based on the unsupported format
                var response = new
                {
                    Code = StatusCodes.Status406NotAcceptable, // HTTP status
code 406
                    ErrorMessage = $"The Requested Format {acceptHeader} is Not
Supported." // Custom error message showing the unsupported format

```

```

};

// Serialize the anonymous object to JSON and write it to the
response body
await
context.Response.WriteAsync(JsonSerializer.Serialize(response));
    }
}
}
}
}

```

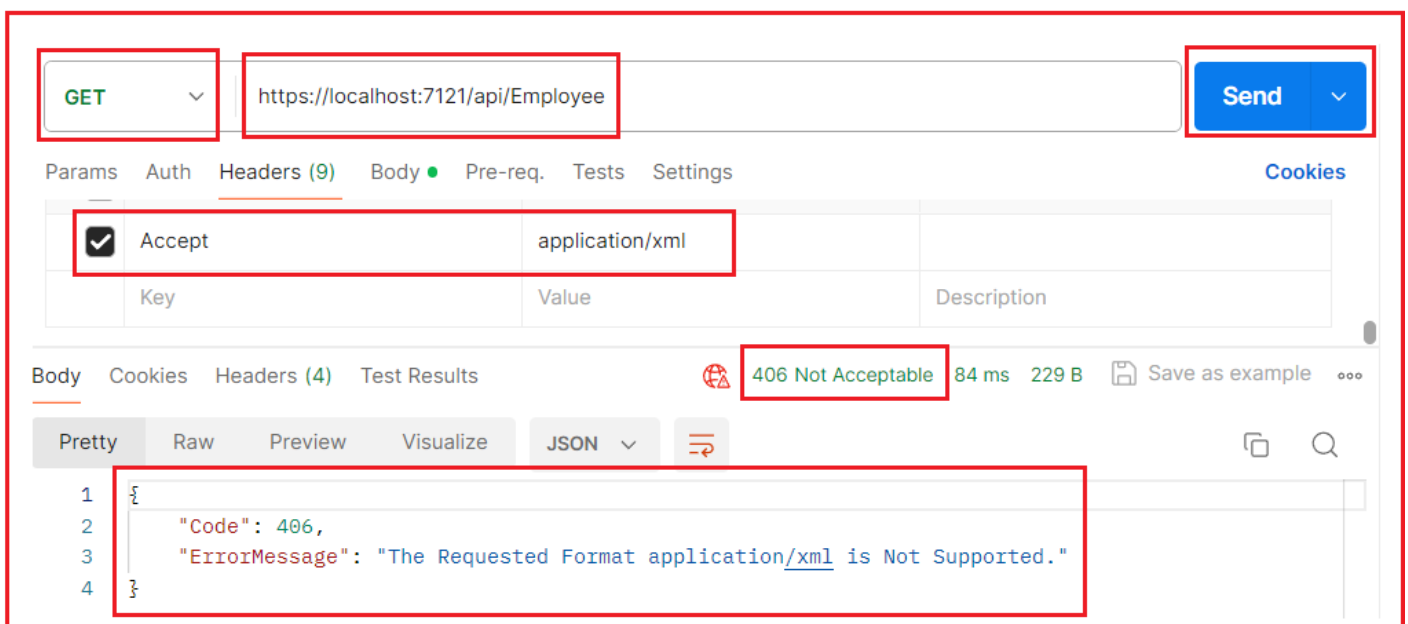
The above custom middleware components check the outgoing responses. If the response status code is **406 Not Acceptable**, it will customize the response accordingly.

Register the Custom Middleware Component

Middleware is registered in the HTTP pipeline using `app.UseMiddleware<T>()` in the Program.cs file. So, please add the following statement before the MapControllers() Middleware Component:

```
app.UseMiddleware<CustomNotAcceptableMiddleware>();
```

Now, with the above Custom Middleware Component, send an HTTP Request with the Accept header value set to application/xml. You will get a **406 Not Acceptable Status Code** along with the **Custom Error Message** as shown in the image below:



The screenshot shows a web browser's developer tools interface. At the top, the request method is **GET** and the URL is **https://localhost:7121/api/Employee**. The **Send** button is visible. Below the URL bar, the **Headers (9)** tab is selected, showing a table with the following content:

Key	Value	Description
☒ Accept	application/xml	

Below the headers, the **Body** tab is selected, showing the response body in JSON format:

```

{
  "Code": 406,
  "ErrorMessage": "The Requested Format application/xml is Not Supported."
}

```

The status bar at the bottom of the developer tools shows the response status code **406 Not Acceptable**, response time **84 ms**, and response size **229 B**. There is also a **Save as example** button.

How Does Content Negotiation Work in ASP.NET Core Web API

Content Negotiation is a mechanism through which the server and client agree on a **Common Data Format** for communication. In ASP.NET Core, this process is handled automatically using the **Accept** and **Content-Type** HTTP

headers and a collection of **Internal Formatters** that know how to read (deserialize) and write (serialize) data in specific formats. Let's break down the process step by step.

Step 1: Client Sends a Request

When a client (such as a browser, mobile app, or another API) sends a request to a Web API endpoint, it typically includes two key headers:

- **Accept Header** → Indicates what **Response Formats** the client can handle (e.g., JSON, XML). For example, Accept: application/json, application/xml
- **Content-Type Header** → Indicates what **format the request body** is in (relevant mainly for POST, PUT, or PATCH requests). For example, Content-Type: application/json

Real-Time Example:

A mobile client sends:

- POST /api/employees
- Content-Type: application/json
- Accept: application/xml
- Body: JSON Data

Here:

- The request body (Content-Type: application/json) is JSON, meaning the client is **sending data in JSON format**.
- The response preference (Accept: application/xml) is XML, meaning the client **wants to receive the server's response in XML**.

Step 2: Server Inspects Request Headers

Once the request reaches the ASP.NET Core middleware pipeline, the **Routing Middleware** identifies the correct **Controller and Action**.

Before invoking your controller's logic, the framework:

- Reads the **Content-Type** header to decide **how to deserialize** the request body (input negotiation).
- Reads the **Accept** header to determine **how to serialize** the outgoing response (output negotiation).

Key Behind-the-Scenes Behavior

- If the request has a **Content-Type** that doesn't match any registered **InputFormatter**, the framework immediately returns **415 Unsupported Media Type**.
- If the Accept header contains unsupported media types and strict negotiation is enabled, ASP.NET Core returns **406 Not Acceptable**.

Step 3: Server Chooses the Best Media Type

ASP.NET Core maintains internal lists of **InputFormatters** and **OutputFormatters**:

- InputFormatters handle incoming data (e.g., JSON → C# object).
- OutputFormatters handle outgoing data (e.g., C# object → XML or JSON).

The framework now tries to find the “best match”:

1. It checks the **client's Accept header** list (which may include multiple media types).
2. It compares them against the **formatters configured** in your app (via `AddControllers()`).
3. The **first supported media type** that matches both client and server capabilities is selected.

Step 4: Request Data Processing (Deserialization)

For requests with a body (e.g., POST, PUT):

1. ASP.NET Core looks at the **Content-Type** header (e.g., `application/json`).
2. It uses the corresponding **InputFormatter** (e.g., `SystemTextJsonInputFormatter`) to convert the raw request body into a .NET object (like `Employee`).
3. If the data format or content type isn't supported, a **415 Unsupported Media Type** error is thrown.

Step 5: Response Generation (Serialization)

Once the controller returns a result (e.g., `Ok(employee)`):

1. ASP.NET Core checks the **Accept** header to decide which **OutputFormatter** to use.
2. It serializes the object (like `Employee`) into the desired format (JSON, XML, etc.).
3. The framework automatically sets the **Content-Type** of the response header to indicate the format being used.

Example:

- If Accept: `application/json` → response Content-Type: `application/json`
- If Accept: `application/xml` and XML formatter is available → response Content-Type: `application/xml`

This makes the API **format flexible**; the same endpoint can serve multiple clients with different data format requirements.

Step 6: Handling Unsupported Formats

If the client requests a format that the server doesn't support:

- **Without strict mode**, ASP.NET Core defaults to the first available formatter (usually JSON).
- **With strict mode (`ReturnHttpNotAcceptable = true`)**, ASP.NET Core returns: HTTP/1.1 406 Not Acceptable

This tells the client that the server doesn't support the requested media type.

Note: Once we discuss the Action Return Types and their execution process, we will discuss the Response Negotiation. I mean, how the .NET Object is deserialized to JSON or XML behind the scenes.

Summary:

Content Negotiation in ASP.NET Core Web API is a powerful mechanism that enhances your API's flexibility and compatibility with different clients. Whether your goal is to provide backward compatibility for legacy XML systems or serve lightweight JSON responses for modern SPAs, ASP.NET Core's formatter-based architecture gives you full control.

In the next article, I will discuss [How to Include and Exclude Properties from Model Binding in ASP.NET Core Web API](#) with Examples. In this article, I will try to explain **Content Negotiation in ASP.NET Core Web API** with examples. I hope you enjoy this article, "Content Negotiation in ASP.NET Core Web API."

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career.

Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

3 thoughts on “Content Negotiation in ASP.NET Core Web API”

CHARITHA

FEBRUARY 20, 2024 AT 2:21 PM

good information

Reply

COYELO

JANUARY 8, 2025 AT 10:34 AM

Typo Here >> <https://dotnettutorials.net/lesson/content-negotiation-in-asp-net-core-web-api/#:~:text=the%20response%20in,XML,-format.%20This%20is>

This should be Json Instead of Xml

Reply

DOT NET TUTORIALS

JANUARY 8, 2025 AT 1:13 PM

Thanks for Identifying the error. We have corrected the same.

Reply

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information